



UNIVERSITETI / UNIVERSITY  
"ISA BOLETINI"  
MITROVICË

# Big Data Processing

...

PYSPARK MACHINE LEARNING - MLLIB

# Machine Learning

In today's data-driven world, the ability to process and analyze massive datasets for machine learning applications is more crucial than ever.

PySpark combines the simplicity of Python with the power of Apache Spark's distributed computing framework, enabling data scientists to perform machine learning tasks on datasets too large for traditional tools.

PySpark's machine learning library offers an impressive range of algorithms and utilities that cover most common machine learning tasks.

PySpark ML supports a wide variety of algorithms across different categories:

- Classification: Logistic regression, decision trees, random forests, gradient-boosted trees, naive Bayes and m-layer perceptron classifiers.
- Regression: Linear regression, generalized linear regression, decision tree regression, random forest regression & gradient-boosted tree regression.
- Clustering: K-means, Gaussian mixture models, Bisecting K-means, and Latent Dirichlet Allocation
- Recommendation: Alternating Least Squares (ALS) for collaborative filtering
- Dimensionality Reduction: Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)
- Feature Engineering: A rich set of tools for feature extraction, transformation, and selection

The PySpark ML library is built around the concept of pipelines, which consist of a sequence of stages. Each stage is either a Transformer or an Estimator

# Quiz

What kind of data structures does pyspark.mllib built-in library support in Spark?

1. Dataframes
2. RDDs
3. Datasets
4. All

Correct: 2

```
1 # Import the library for ALS
2 from pyspark.mllib.recommendation import ALS
3
4 # Import the library for Logistic Regression
5 from pyspark.mllib.classification import LogisticRegressionWithLBFGS
6
7 # Import the library for Kmeans
8 from pyspark.mllib.clustering import KMeans
```

# Announcement: DataFrame-based API is primary API

The MLlib RDD-based API is now in maintenance mode.

As of **Spark 2.0** , the **RDD-based APIs** in the `spark.mllib` package have **entered maintenance mode** . The **primary Machine Learning API** for **Spark** is now the **DataFrame-based API** in the `spark.ml` package.

Is MLlib deprecated?

No. MLlib includes both the RDD-based API and the DataFrame-based API. The RDD-based API is now in maintenance mode. But neither API is deprecated, nor MLlib as a whole.

# Example 1: Loading Movie Lens - Part 1

Collaborative filtering is a technique for recommender systems wherein users' ratings and interactions with various products are used to recommend new ones. With the advent of Machine Learning and parallelized processing of data, recommender systems have become widely popular in recent years, and are utilized in a variety of areas including movies, music, news, books, research articles, search queries, social tags. In this 3-part exercise, your goal is to develop a simple movie recommendation system using PySpark MLlib using a subset of [MovieLens 100k dataset](#).

- Load the ratings.csv dataset into an RDD.
- Split the RDD using , as a delimiter.
- For each line of the RDD, using Rating() class create a tuple of userID, productID, rating.
- Randomly split the data into training data and test data (0.8 and 0.2).

```
1 # Load the data into RDD
2 data = sc.___(file_path)
3
4 # Split the RDD
5 ratings = data.___(lambda l: l.split('___'))
6
7 # Transform the ratings RDD
8 ratings_final = ratings.___(lambda line: Rating(int(line[0]), int(___), float(___)))
9
10 # Split the data into training and test
11 training_data, test_data = ratings_final.___([0.8, 0.2])
```

# Example 1: Model training and predictions - Part 2

After splitting the data into training and test data, in the second part of the exercise, you'll train the ALS algorithm using the training data. PySpark MLlib's ALS algorithm has the following mandatory parameters - rank (the number of latent factors in the model) and iterations (number of iterations to run). After training the ALS model, you can use the model to predict the ratings from the test data. For this, you will provide the user and item columns from the test dataset and finally return the list of 2 rows of predictAll() output.

- Train ALS algorithm with training data and configured parameters (rank = 10 and iterations = 10).
- Drop the rating column in the test data, which is the third column.
- Test the model by predicting the rating from the test data.
- Return a list of two rows of the predicted ratings.



```
1 # Create the ALS model on the training data
2 model = ALS.____(____, rank=10, iterations=10)
3
4 # Drop the ratings column
5 testdata_no_rating = test_data.__(lambda p: (p[0], ____))
6
7 # Predict the model
8 predictions = model.____(testdata_no_rating)
9
10 # Return the first 2 rows of the RDD
11 predictions.____(2)
```

# Example 1: Model Evaluation - Part 3

## Model evaluation using MSE

After generating the predicted ratings from the test data using ALS model, in this final part of the exercise, you'll prepare the data for calculating Mean Square Error (MSE) of the model. The MSE is the average value of  $(\text{original rating} - \text{predicted rating})^2$  for all users and indicates the absolute fit of the model to the data.

To do this, first, you'll organize both the ratings\_final and predictions RDDs to make a tuple of ((user, product), rating)). In both RDDs the mapping is:

**0: user & 1: product & 2: rating**

Then you'll join transformed RDDs and finally apply a squared difference function along with mean() to get the MSE. Instructions:

- Organize ratings RDD to make ((user, product), rating).
- Organize predictions RDD to make ((user, product), rating).
- Join the prediction RDD with the ratings RDD.
- Evaluate the model using MSE between original rating and predicted rating and print it.

```
1 # Prepare ratings data
2 rates = ratings_final.__(lambda r: ((r[0], r[1]), __))
3
4 # Prepare predictions data
5 preds = predictions.__(lambda r: ((__, __), __))
6
7 # Join the ratings data with predictions data
8 rates_and_preds = rates.__(preds)
9
10 # Calculate and print MSE
11 MSE = rates_and_preds.__(lambda r: (r[1][0] - r[1]__)**2).mean()
12 print("Mean Squared Error of the model for the test data = {:.2f}".format(__))
```



UNIVERSITETI / UNIVERSITY  
"ISA BOLETINI"  
MITROVICË

# Stay Curious. Questions?

Thank you for your attention.  
Lorent Sinani



Scan to connect on LinkedIn